

Keywords: heterogeneous hardware • subquadratic attention
• disaggregation • energy efficiency • LLM serving

Rethinking Heterogeneous System Disaggregation for Subquadratic Attention

Prefill-Decode Disaggregation Misses Within-Decode Arithmetic Intensity Heterogeneity — Fine-Grained SQD Delivers 31–56% Tokens-per-Joule Gains

By Arya Tschand, Yaosheng Fu, Vikram Sharma Mailthody, Nicolai Oswald, Po-An Tsai, Ritchie Zhao, Oreste Villa, Vijay Janapa Reddi, Karu Sankaralingam (Harvard; NVIDIA; UW-Madison)

arXiv:2609.13134

Frontier LLMs increasingly deploy subquadratic attention — sparse, linear, and sliding-window variants — to reduce the $O(L)$ KV memory footprint of long contexts. Heterogeneous hardware systems already disaggregate by phase: prefill (compute-bound) on one device, decode (memory-bound) on another. But this coarse split ignores a critical observation: within a single decode step, different operators have sharply different arithmetic intensity profiles. SQD (SubQuadratic Disaggregation) exploits this by splitting the decode step itself, routing memory-bound and compute-bound sub-phases to the hardware optimized for each.

AT A GLANCE

Problem: Prefill-decode disaggregation does not account for within-decode operator heterogeneity in models with subquadratic attention.

Why it matters: 31–56% energy efficiency gain is left on the table on frontier models (GLM 5.2, Nemotron 3 Ultra, Gemma 4 31B).

Method: SQD splits decode by sub-phase: top- k selection (memory device) vs. top- k attention + FFN (compute device) for sparse; dense layers vs. subquadratic + FFN for linear models.

Result: 53%, 31%, 56% tokens/J improvement on GLM 5.2, Nemotron 3 Ultra, Gemma 4 31B on an 8×B200 system.

Evidence: Energy and throughput measured on production model weights; ablations by disaggregation level.

The Big Picture

Full attention has arithmetic intensity $I = \text{FLOPs/bytes} \approx d/2$ (roughly compute-bound for large d). Decode batch size 1 is memory-bound ($I \approx 1$) regardless of attention type. The roofline model predicts runtime:

$$T(O) = \max\left(\frac{F}{C}, \frac{B}{\beta}\right)$$

where C (FLOP/s) and β (bytes/s) are device peaks. Routing to the device where this bound is tightest minimizes runtime and energy per token.

Why Existing Approaches Fall Short

Current disaggregation schemes split at the phase boundary: prefill (high compute intensity, batch = L) goes to GPU; decode (low intensity, batch = 1–few) goes to memory-optimized accelerator. This is appropriate for dense attention. For subquadratic attention, the decode step itself contains operators with radically different intensities:

Operator	I (FLOP/byte)	Bound
Top- k KV selection	≈ 1	Memory
Top- k attn. + FFN	$\approx 10^3$	Compute
Dense attn. layer	≈ 10	Memory
Subquad. layer + FFN	≈ 2	Memory

Routing the entire decode step to a memory-optimized device wastes compute capacity on operators that are actually compute-bound.

Core Mechanism: SQD

SQD for sparse attention. Each decode step is split into two sub-phases:

1. *Selection:* Traverse all L keys to compute relevance scores and select top- k entries. Cost: $O(L \cdot d_{\text{head}})$ reads, $O(L)$ FLOPs. $I \approx 1$ FLOP/byte $\rightarrow$ memory device.
2. *Attention + FFN:* Attend to the selected k entries and run the FFN. With $k \ll L$, KV footprint is small and static; $I \approx 10^3$ FLOP/byte $\rightarrow$ compute device.

SQD for linear/sliding-window attention. Split by layer type:

1. *Dense attention layers:* Memory-bound at decode batch 1 $\rightarrow$ memory device.
2. *Subquadratic layers + FFN:* Linear attention state update is compute-bound $\rightarrow$ compute device.

Communication cost: The two sub-phases exchange activation tensors of size $O(d)$ per token per layer — small compared with $O(L \cdot d)$ KV data transfer.

Technical Formulation

The **ridge point** of a device is:

$$I^* = \frac{C}{\beta}$$

On B200: $C = 4.5 \times 10^{15}$ FP8 FLOP/s, $\beta = 8 \times 10^{12}$ bytes/s, so $I^* \approx 562$ FLOP/byte.

Routing criterion: Assign operator O to the memory-optimized device if $I(O) < I^*$, otherwise to the compute-optimized GPU.

Energy per token: Routing each operator to its optimal device reduces average power draw per operation; the total energy per token is:

$$E = \sum_O \frac{F_O}{P_O} + \frac{B_O}{P_O^{\text{mem}}}$$

where P_O is the TDP of the assigned device.

Experimental Results

Evaluated on an 8×B200 heterogeneous system (4 compute-optimized + 4 memory-bandwidth-optimized) with production model weights:

Model	Attn. Type	Tokens/J Gain
GLM 5.2	Sparse	+53%
Nemotron 3 Ultra	Linear	+31%
Gemma 4 31B	Sliding-win	+56%

Throughput gains (tokens/s, batch = 8, seq = 32k):

Model	GPU-only	SQD	Speedup
GLM 5.2	1,840	2,710	1.47×
Gemma 4 31B	2,190	3,080	1.41×

Ablations and Interpretation

Selection-only disaggregation: Routing only the top- k selection phase accounts for 68% of the total energy gain for sparse attention models.

Without within-decode layer-type split: Routing by prefill/decode phase only reduces energy gain from 53% to 29% for GLM 5.2, quantifying the value of intra-decode disaggregation.

Communication overhead: Cross-device activation transfer adds 7–12% latency but is offset by 40–55% reduction in time-per-token on memory-bound operators.

Dense models: SQD provides no benefit when all attention layers are dense — standard prefill-decode disaggregation is already optimal in that case.

Reference

Arya Tschand et al. **Rethinking Heterogeneous System Disaggregation for Subquadratic Attention**. arXiv:2609.13134, September 2026. <https://arxiv.org/abs/2609.13134>